

Eficiência Algorítmica na Recuperação de Dados: Uma Análise Comparativa entre Busca Sequencial e Tabela Hash com Encadeamento

Ana Luiza Gomes de Lima¹, Lucas Pessoa Ranieri², Pedro Augusto Machado Cardoso³

Curso de Ciência da Computação – Centro Universitário Instituto de Educação Superior de Brasília (IESB)
Brasília – DF – Brazil

ana.lima32@iesb.edu.br, lucas.ranieri@iesb.edu.br,
pedro.a.cardoso@iesb.edu.br

Abstract. *This paper presents a comparative performance analysis between two data structures applied to record retrieval in large-scale data scenarios: the Dynamic Vector (Sequential Search) and the Hash Table with external chaining collision handling. Using a real-world dataset containing 300,006 products, an automated experimental protocol was implemented to measure the average recovery time across four strategic scenarios (Beginning, Middle, End, and Non-existent). Empirical results revealed that the Hash Table mitigated linear time growth, operating in constant time $O(1)$ and generating only 1 collision across the entire data domain. In the worst-case scenario of the linear search, the hashing approach reduced the search time by more than 710,000 times, proving to be indispensable for high-performance systems.*

Resumo. *Este artigo apresenta uma análise comparativa de desempenho entre duas estruturas de dados aplicadas à busca de registros em cenários de grande volumetria: o Vetor Dinâmico (Busca Sequencial) e a Tabela Hash com tratamento de colisões por encadeamento externo. Utilizando um dataset real contendo 300.006 produtos, foi implementado um protocolo experimental automatizado para medir o tempo médio de recuperação em quatro cenários estratégicos (Início, Meio, Fim e Inexistente). Os resultados empíricos revelaram que a Tabela Hash mitigou o crescimento linear de tempo, operando em tempo constante $O(1)$ e gerando apenas 1 colisão em todo o domínio de dados. No pior cenário da busca linear, a abordagem via hashing reduziu o tempo de busca em mais de 710.000 vezes, demonstrando-se indispensável para sistemas de alta performance.*

1. Introdução

A eficiência na recuperação de informações constitui um dos pilares críticos no desenvolvimento de sistemas de software modernos, especialmente diante do crescimento exponencial do volume de dados manipulados por aplicações corporativas. A escolha da estrutura de dados adequada impacta diretamente o consumo de recursos computacionais, a escalabilidade do sistema e o tempo de resposta oferecido ao usuário final.

Este trabalho documenta o ciclo de desenvolvimento completo de um sistema de gerenciamento de produtos, estruturado em duas fases complementares de engenharia de software. Na Fase I, o sistema foi concebido utilizando uma abordagem tradicional baseada em vetores dinâmicos. Embora essa estrutura ofereça simplicidade de implementação e acesso direto via índice, a operação de busca por chaves não ordenadas exige uma varredura linear. Essa estratégia possui complexidade computacional de pior caso classificada como $O(n)$, o que gera um gargalo de desempenho severo à medida que o volume de registros escala para a ordem de centenas de milhares. Para mitigar essa limitação, a Fase II do projeto consistiu na transição arquitetural para uma Tabela Hash com tratamento de colisões por encadeamento. O objetivo principal deste artigo é apresentar uma análise comparativa e detalhada entre essas duas fases. Através de um protocolo experimental automatizado, busca-se medir o impacto real da migração de estruturas e validar empiricamente as asserções teóricas que posicionam o hashing como uma solução de tempo constante, $O(1)$, indispensável para a recuperação de registros em cenários de alta volumetria.

2. Fundamentação Teórica

A análise de algoritmos apoia-se na notação assintótica para classificar o consumo de tempo e espaço de um algoritmo de forma independente do hardware utilizado. Na análise de pior caso, a Busca Sequencial operando em um arranjo não ordenado exige que cada elemento seja inspecionado consecutivamente até que a chave correspondente seja localizada ou que o fim da estrutura seja alcançado. Consequentemente, para um conjunto contendo n elementos, a função que descreve o número de comparações cresce linearmente, caracterizando uma complexidade temporal de pior caso de classe $O(n)$.

Por outro lado, as tabelas de dispersão baseiam-se na aplicação de uma função matemática, denominada função hash, sobre a chave de busca do registro. O valor resultante dessa função determina diretamente o índice da memória onde o registro está (ou deveria estar) armazenado. No cenário teórico perfeito, o tempo de acesso a qualquer posição da tabela é feito de maneira direta, configurando uma complexidade de tempo constante de classe $O(1)$.

Contudo, como o domínio de chaves possíveis frequentemente supera o tamanho físico disponível na tabela (fenômeno conhecido como Princípio da Casa dos Pombos), diferentes chaves podem mapear para o mesmo índice de memória, gerando uma colisão. Para contornar este problema, a literatura especializada propõe diferentes táticas, destacando-se o encadeamento externo ou encadeamento em nós separados.

Nesta estratégia, cada bucket (posição da tabela) não guarda o registro diretamente, mas sim um ponteiro para o nó inicial de uma lista encadeada dinâmica. Quando ocorre uma colisão, o novo elemento é simplesmente anexado à lista daquela respectiva posição, preservando a integridade e a acessibilidade de todos os dados inseridos.

3. Metodologia e Desenvolvimento das Fases

O desenvolvimento experimental e a coleta de dados foram conduzidos em um ambiente virtualizado controlado (VS Code Dev Container), provido de um compilador GCC (GNU Compiler Collection), utilizando flags de depuração e conformidade padrão ANSI C. A fonte primária de dados constituiu-se de um arquivo em formato texto CSV contendo 300.006 registros exclusivos de produtos de tecnologia, onde cada linha representava uma entidade estruturada composta por um ID numérico inteiro, uma cadeia de caracteres descritiva (nome do produto) e um valor monetário de ponto flutuante.

3.1 Fase I: Vetor Dinâmico e Busca Sequencial

Na Fase I, o sistema foi estruturado em torno de um Tipo Abstrato de Dado (TAD) para representar a entidade Produto. Em termos de implementação na linguagem C, a persistência temporária baseou-se em um vetor dinâmico gerenciado por ponteiros nativos. A arquitetura de controle do vetor foi encapsulada em uma estrutura principal que continha três campos fundamentais: um ponteiro do tipo Produto para apontar para o bloco de memória contíguo, um inteiro de controle de tamanho (indicando quantos registros válidos estavam preenchidos) e um inteiro marcador de capacidade máxima alocada.

O processo de carga foi desenhado para ler o arquivo CSV linha por linha utilizando fluxos de entrada (*fopen e fgets*). À medida que cada registro era parseado, o programa realizava a inserção na última posição vaga do arranjo. O gerenciamento de memória foi um ponto crítico: quando o limite de alocação da capacidade era atingido, o sistema invocava o comando de realocação dinâmica de memória (*realloc*), expandindo o tamanho físico do vetor em blocos modulares para acomodar os novos dados, garantindo que nenhum registro fosse perdido por estouro de memória (*buffer overflow*).

O algoritmo de busca associado a esta fase foi implementado por meio de um laço de repetição iterativo tradicional (*for*). Este laço comparava o ID solicitado pelo usuário com o ID armazenado em cada posição de memória do vetor, estendendo-se obrigatoriamente do índice zero até o índice final do vetor. Caso o elemento estivesse na última posição ou não existisse, o algoritmo executava exatamente n comparações antes de retornar o controle ao fluxo principal do programa.

3.2 Fase II: Tabela Hash com Listas Encadeadas Externas

Na Fase II, visando erradicar o gargalo linear de desempenho verificado na fase anterior, o sistema sofreu uma completa reengenharia em sua arquitetura de dados em memória.

O arquivo de cabeçalho (.h) e o arquivo de implementação correspondente foram inteiramente reconfigurados para suportar o conceito de listas encadeadas externas (*chaining*) acopladas a uma tabela de ponteiros.

Cada elemento da tabela passou a ser tratado como um *bucket* que aponta para um nó de lista encadeada dinâmica. A estrutura do nó (*struct Node*) foi projetada contendo dois campos: os dados encapsulados do tipo Produto e um ponteiro de autorreferência apontando para o próximo nó da sequência (*struct Node* proximo*).

Em conformidade estrita com os critérios especificados no projeto, a Tabela Hash foi alocada dinamicamente na memória (através de *malloc*) com o tamanho (*m*) configurado para ser exatamente correspondente ao volume total da massa de dados lida, estabelecendo $m = 300.006$ posições de *buckets*. Esta parametrização cirúrgica resultou em um fator de carga de projeto ideal igual a um ($\alpha = 1,0$).

A função de dispersão (*hash*) adotada seguiu a formulação clássica do método da divisão para chaves inteiras, descrita matematicamente pela equação:

$$h(ID) = ID \bmod m$$

Onde "*mod*" representa o operador de resto da divisão inteira (operador % em C). Durante o processo de carga dos 300.006 registros, para cada produto lido do CSV, a função $h(ID)$ calculava instantaneamente o índice correspondente dentro do intervalo $[0, m-1]$.

O tratamento de colisões foi integrado diretamente na lógica de inserção: caso o bucket calculado estivesse nulo (vazio), o nó era alocado e inserido ali diretamente. Se o índice já contivesse um elemento previamente cadastrado (caracterizando uma colisão), o sistema realizava o tratamento inserindo o novo nó no início da lista encadeada daquela posição (inserção na cabeça da lista), ajustando os ponteiros de forma que o novo nó apontasse para o antigo primeiro elemento. Esta abordagem garantiu que o custo de inserção em memória permanecesse fixado em tempo constante, evitando a varredura das listas durante a carga do arquivo.

3.3 Protocolo Experimental de Aferição Temporal

Para avaliar comparativamente a eficiência das duas fases e mitigar discrepâncias estatísticas induzidas pelo escalonamento de processos do sistema operacional, desenhou-se um protocolo experimental automatizado e rigoroso. O programa foi codificado para realizar quatro baterias de testes focando em identificadores chaves (IDs) estrategicamente pré-selecionados na base de dados para representar diferentes cenários assintóticos:

- **INÍCIO:** ID 43614, localizado no primeiríssimo registro do arquivo CSV original, mapeado no índice zero do vetor dinâmico.
- **MEIO:** ID 3341, localizado no registro de número 150.003, representando fielmente o centro geométrico da massa de dados.
- **FINAL:** ID 189553, localizado na última linha útil do arquivo (registro de número 300.006).

- **INEXISTENTE:** ID -9999, uma chave artificial que sabidamente não pertencia ao conjunto de dados, desenhada especificamente para forçar ambos os algoritmos a avaliarem as estruturas até os seus limites físicos absolutos de pior caso.

O mecanismo de medição temporal utilizou bibliotecas de alta precisão do sistema operacional. Cada teste individual de cenário não mediu apenas uma busca isolada, o que poderia gerar imprecisões devido à resolução do clock do processador. Em vez disso, o protocolo executou um bloco automatizado de 1.000 buscas consecutivas e idênticas para o mesmo ID.

Este bloco de mil buscas foi repetido de forma independente por 3 vezes consecutivas (Repetições 1, 2 e 3) para cada estrutura avaliada. Ao término de cada bloco, o tempo total acumulado em milissegundos (ms) foi registrado pelo sistema. O tempo médio final de uma única busca foi calculado computacionalmente dividindo a média do tempo acumulado das três repetições pelo fator das 1.000 operações simuladas.

4. Resultados e Discussão

4.1 Eficiência de Dispersão e Métrica de Qualidade

Um dos resultados mais expressivos e academicamente relevantes obtidos na Fase II residiu na métrica de colisão gerada pela função *hash* desenvolvida. Ao carregar a totalidade dos 300.006 produtos de tecnologia, o sistema registrou o número surpreendente de apenas 1 colisão em todo o domínio de dados.

Este comportamento numérico indica que a função de *hash* baseada no método da divisão operou com uma distribuição de chaves que se aproximou perfeitamente do conceito de Dispersão Uniforme Simples. Na engenharia de *software*, obter apenas uma colisão em um universo de mais de trezentos mil registros significa que a Tabela *Hash* distribuiu os elementos de forma quase bijetiva pela memória.

Praticamente todos os 300.006 *buckets* alocados contiveram listas encadeadas com comprimento máximo igual a 1. Consequentemente, o custo de percorrer os nós internos das listas encadeadas durante as buscas foi mitigado a zero na prática. Este resultado eliminou a possibilidade de degradação da tabela para o pior caso teórico (onde todas as chaves colidiriam em um mesmo *bucket*, transformando a tabela em uma lista linear), preservando o cenário ideal de eficiência computacional.

4.2 Análise Temporal Comparativa e Fenômenos de Hardware

Os dados de desempenho coletados diretamente no terminal da aplicação durante a execução do protocolo experimental automatizado foram compilados de forma estruturada. A Tabela 1 apresenta a comparação direta entre os tempos médios calculados para cada cenário e o respectivo ganho de desempenho obtido.

Tabela 1. Comparativo de tempo médio por busca e fator de aceleração empírico

Cenário de Busca	ID Alvo	Busca Sequencial (ms)	Tabela Hash (ms)	Fator de Aceleração (speedup)
INÍCIO	43614	0,000004	0,000005	~1,00x (Equivalente)
MEIO	3341	1,572596	0,000007	224.656x mais rápida
FINAL	189553	3,551487	0,000005	710.297x mais rápida
INEXISTENTE	-9999	3,412290	0,000006	568.715x mais rápida

A análise aprofundada dos dados expressos na Tabela 1 revela uma conformidade absoluta com os preceitos teóricos da ciência da computação e da análise de algoritmos. A Busca Sequencial implementada na Fase I exibiu com clareza a rampa de crescimento linear característica de algoritmos pertencentes à classe de complexidade $O(n)$.

No cenário **INÍCIO**, onde o ID alvo encontrava-se na primeira posição física do vetor, o laço de repetição foi interrompido imediatamente após a primeira comparação bem-sucedida, registrando um tempo médio quase nulo de 0,000004 milissegundos. Contudo, quando a chave de busca foi deslocada para o centro geométrico da estrutura (cenário **MEIO**), o algoritmo foi obrigado a processar sequencialmente cerca de 150.000 registros na memória, fazendo com que o tempo médio saltasse acentuadamente para 1,572596 ms.

O pico de degradação temporal da busca sequencial manifestou-se no cenário **FINAL**, alcançando a marca de 3,551487 ms. Este valor reflete o custo computacional máximo do algoritmo, que precisou realizar todas as 300.006 iterações e comparações em memória para localizar o último produto do conjunto.

No cenário **INEXISTENTE**, a varredura linear registrou a média de 3,412290 ms. Sob a ótica da arquitetura de computadores, essa pequena variação decorre de otimizações nativas de *hardware*, como os mecanismos de previsão de desvio (*branch prediction*) da CPU e o reaproveitamento de páginas de dados nas linhas de cache L1/L2 do processador durante as baterias consecutivas de repetições em que o algoritmo falha continuamente. O cenário inexistente consolida o alto custo da Fase I, demonstrando que o sistema é obrigado a realizar uma varredura completa em 100% das vezes em que uma busca falha.

Em contrapartida, a Tabela *Hash* desenvolvida na Fase II demonstrou estabilidade plena, imunidade locacional e total independência em relação à posição original ou à existência do registro no sistema. Os tempos médios registrados pela estrutura de dispersão mantiveram-se confinados de maneira homogênea na faixa

estritamente compreendida entre 0,000005 ms e 0,000007 ms em todas as simulações executadas.

Esta regularidade cronométrica comprova empiricamente a teoria da complexidade de tempo constante $O(1)$. O tempo necessário para efetuar o cálculo aritmético da função de dispersão e acessar diretamente o ponteiro do *bucket* correspondente na memória *RAM* permanece idêntico, independentemente de o ID estar localizado na frente, no meio, na última linha do arquivo CSV, ou se sequer constava na base de dados.

O cálculo do *Speedup* (Fator de Aceleração) evidencia o impacto prático massivo trazido pela reforma arquitetural da Fase II. Dividindo-se o tempo gasto pela varredura linear pelo tempo consumido pela indexação por hash, constata-se que a Tabela *Hash* foi mais de 224 mil vezes mais veloz para localizar o elemento central e alcançou uma vantagem extraordinária de 710.297 vezes mais rapidez na recuperação do produto posicionado no final da estrutura. O sistema reduziu uma operação que demorava milissegundos para a escala de nanosegundos, tornando o tempo de resposta imperceptível.

5. Conclusão

A transição arquitetural e a evolução de engenharia promovidas entre a Fase I e a Fase II deste projeto foram concluídas com êxito absoluto, atendendo a todos os requisitos técnicos, funcionais e de desempenho estipulados. A substituição do modelo linear de vetores por uma infraestrutura baseada em tabelas de dispersão com encadeamento externo provou ser a estratégia definitiva para eliminar os problemas de latência associados à manipulação de grandes massas de informações em memória.

Os dados estatísticos coletados por meio do protocolo experimental ratificaram de maneira inequívoca as previsões teóricas de complexidade computacional. Enquanto o vetor dinâmico apresentou forte dependência locacional e uma degradação temporal severa de ordem $O(n)$, a Tabela *Hash* preservou sua eficiência operacional em tempo constante $O(1)$ em todos os cenários.

O clássico *trade-off* (relação de compromisso) de engenharia de *software* — representado neste projeto pelo consumo ligeiramente superior de memória *RAM* necessário para alocar as estruturas de ponteiros e nós dinâmicos das listas encadeadas — mostrou-se totalmente justificado e amplamente compensado pelo ganho de velocidade obtido, o qual ultrapassou a escala de 710.000 vezes de aceleração no pior caso.

Conclui-se, portanto, que para o projeto de sistemas de *software* modernos, escaláveis e capazes de operar sob regime de alta concorrência ou em tempo real, o uso de tabelas de dispersão devidamente dimensionadas e associadas a funções de hash uniformes constitui uma decisão de projeto mandatória, transformando gargalos lineares em operações instantâneas de alta performance.

Referências

- [CELES, W.; CERQUEIRA, R.; RANGEL, J. L. (2016)]. *Estruturas de Dados*. 2. ed. Rio de Janeiro: Elsevier.
- [CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. (2012)]. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier.
- [FEOFILOFF, P. (2009)]. *Algoritmos em Linguagem C*. Rio de Janeiro: Elsevier.
- [TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. (1995)]. *Estruturas de Dados Usando C*. São Paulo: Makron Books.
- [ZIVIANI, N. (2011)]. *Projeto de Algoritmos: Com Implementações em Pascal e C*. 3. ed. São Paulo: Cengage Learning.